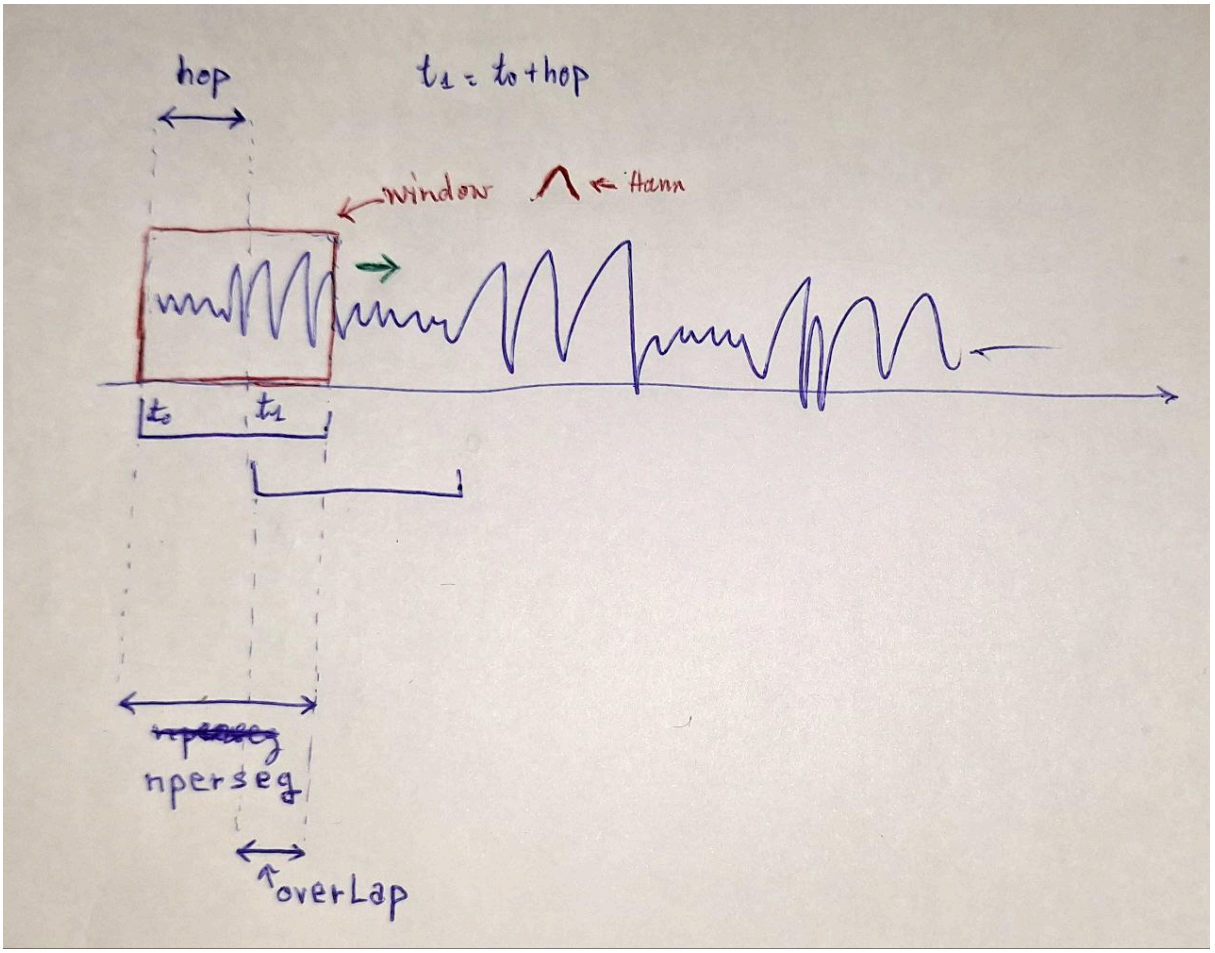
AutoDecompositon: STFT CUSTOM

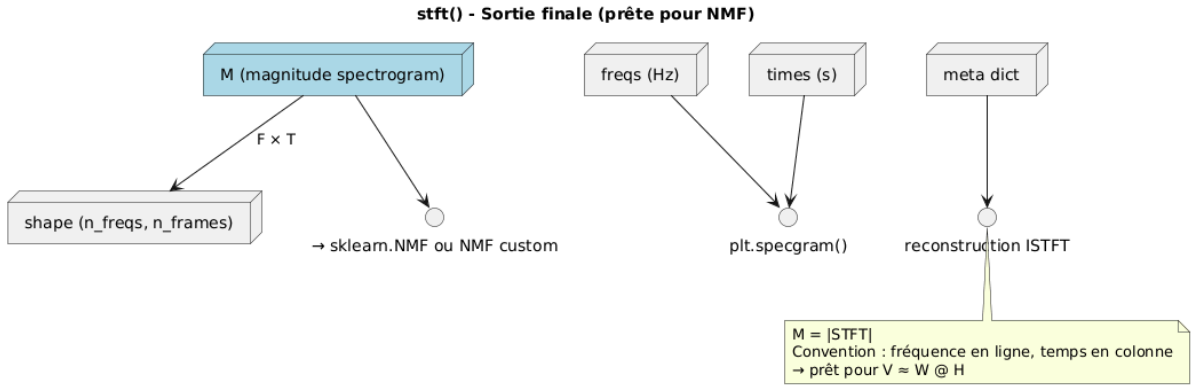


stft

```
x(t)
  ↓ framing() + padding
  ↓ get_window("hann")
  ↓ apply_window() → atténuation bords
  ↓ fft_frames() → STFT complexe
  ↓ np.abs() → magnitude M
  ↓ (F × T), freqs, times, meta
      ↓
    NMF(M ≈ W @ H)
```

↓

Reconstruction sources (Fig. 4 sujet)

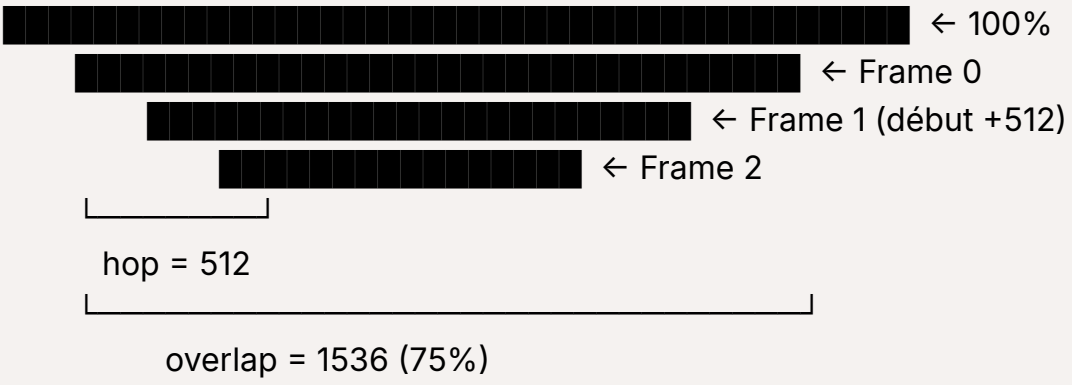


compute_hop()

Objectif : Calculer hop = distance entre deux débuts de frame
Standard NMF : hop_fraction = 0.25 → 75% overlap

Exemple:
nperseg = 2048
hop_fraction = 0.25
hop = nperseg × hop_fraction = 512
overlap = 1 - hop/nperseg = 75%

Fenêtre (nperseg = 2048)



framing()

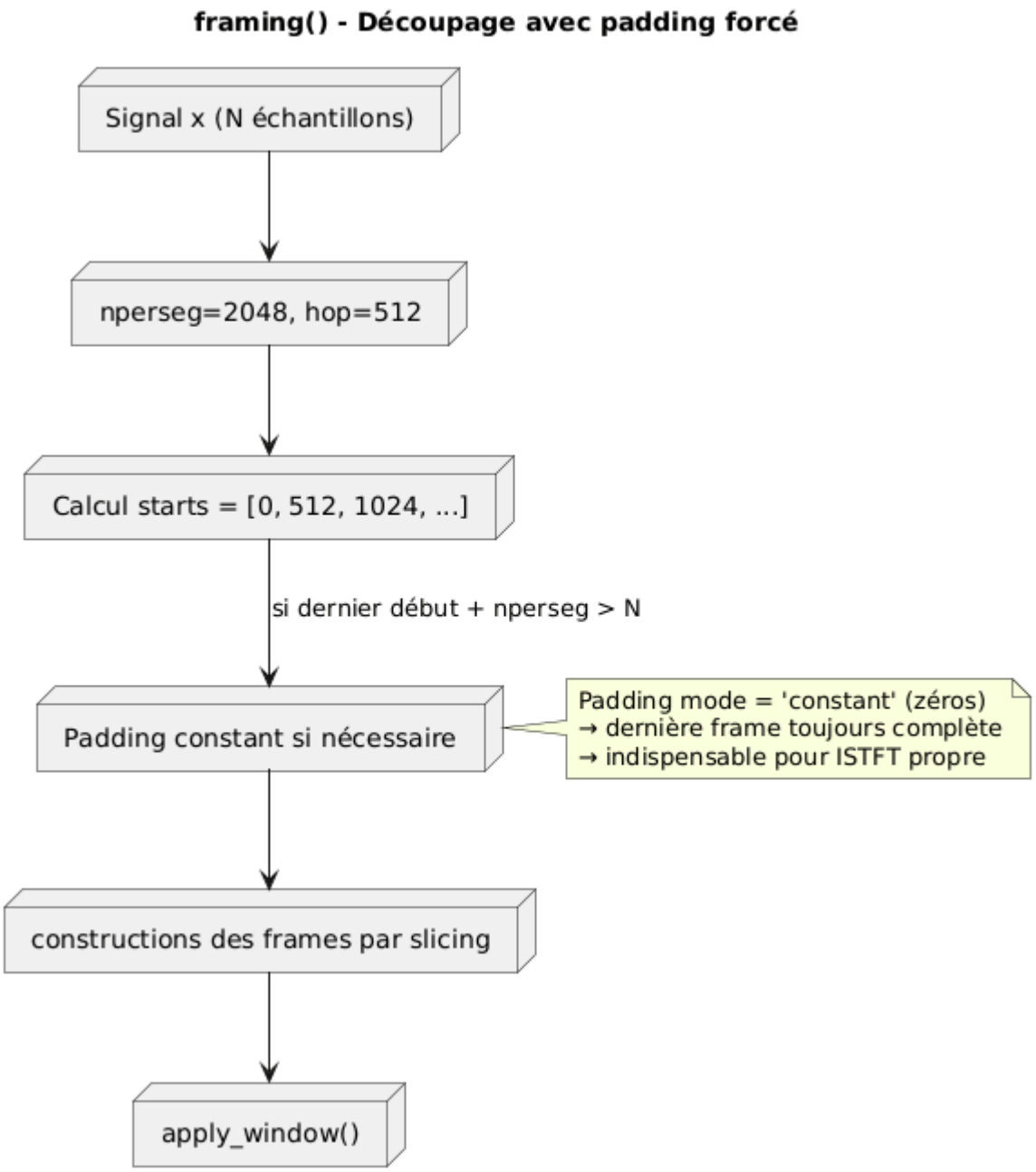
Découpe le signal en frames avec padding constant à la fin
Garantit au moins 1 frame même si signal court

image plantuml

Signal original (ex: N=5000)
x[n] : 

Après padding → longueur = 6144 (exemple)
x_pad:  000000000

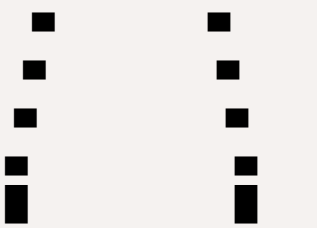
Frames extraites :
Frame 0 : [x[0]   → x[2047]]
Frame 1 : [x[512] → x[2559]]
Frame 2 : [x[1024] → x[3071]]
...
Frame 11: [x[5632] → x[7679]] ← contient beaucoup de zéros



get_window()

Objectif : retourne la fenetre de Hann (par defaut) :
w[n] = 0.5 - 0.5*cos(2πn/(nperseg-1))
Fenêtre Hann (nperseg=2048)





apply_window()

Multiplication élément par élément

Frame i : [x0, x1, x2, ..., x2047]
Window : [w0, w1, w2, ..., w2047]
↓ élément par élément
Resultat: [x0*w0, x1*w1, ..., x2047*w2047]
→ atténue les bords → évite fuites spectrales

fft_frames()

Vous connaissez déjà, on fait la fft sur la frame.

magnitude_or_power()

Pour NMF → on travaille sur |STFT| (magnitude) Pas sur puissance (|STFT|²) ni phase